7. Structs, Methods, and Pointers

07. Structs, Methods, and Pointers

Phase: Phase 1 — CS fundamentals through Go
Estimated time: 1–2 days
Slug: `07-structs-methods-pointers`

Learning objectives

- Define structs and compose them with embedding
- Choose value vs pointer receivers deliberately
- Explain what a pointer stores (an address)
- Avoid nil pointer dereferences

Prerequisites

Complete prior numbered lessons. Type every example yourself — reading Go is not writing Go.

Why this matters

Structs are Go's objects-without-classes. Pointer vs value receivers decide mutability and copies — foundational for APIs and concurrency safety.

This lesson is a building block for later modules in the Goodway path. If you only skim, the exercises will feel like busywork; if you type the examples and narrate them aloud, the same patterns will reappear in HTTP handlers, concurrency, and AI service glue.

Core concepts

- Structs group fields; prefer keyed composite literals.
- Pointers store addresses; `&u` takes address, `*p` dereferences.
- Pointer receivers mutate; value receivers operate on copies.
- If any method needs *T, prefer* T consistently for that type.
- Embedding promotes fields/methods — composition over inheritance.

Hold these in working memory while you code. Each bullet should map to something you can demonstrate with a three-line program or a failing compile error.

Worked Go examples

Create a scratch module and type this yourself (do not paste blindly):

```
mkdir -p /tmp/gw-structs, && cd /tmp/gw-structs,
go mod init example.com/scratch
```

Then create `main.go`:

```
package main
```

```go
import "fmt"

type Counter struct{ n int }

func NewCounter(start int) *Counter { return &Counter{n: start} }
func (c *Counter) Inc()             { c.n++ }
func (c *Counter) Value() int       { return c.n }

func main() {
    c := NewCounter(2)
    c.Inc()
    fmt.Println(c.Value())
}
```

Run it:

```
go run .
```

Expected output:

```
3
```

If your output differs, diff character-by-character — especially whitespace and types. Then change one line deliberately and predict the new output before running.

Mini-lab (`go run` steps)

1. Implement Counter; verify Inc mutates.
2. Change Inc to a value receiver and observe the bug.
3. Embed struct B inside A and call a promoted method.
4. Print a nil *Counter method call carefully (expect panic if deref).

Write down one sentence explaining *why* the output matches the code before you move on. Time-box the lab to 15–25 minutes; struggle is part of encoding the skill.

Curriculum notes (expanded from the compact draft)

Structs

```go
type User struct {
    Name string
    Age  int
}

u := User{Name: "Ada", Age: 36}
```

Pointers

A pointer holds a **memory address** of a value.

```go
p := &u       // *User
p.Age = 37    // Go auto-dereferences
```

Use pointers when: - you need to mutate the caller's struct - the struct is large and you want to avoid copies - you need `nil` to mean "absent"

Methods

```
func (u User) Greet() string { return "hi " + u.Name }
func (u *User) Birthday() { u.Age++ }
```

Rule of thumb: if any method needs a pointer receiver, be consistent for the type. Value receivers get a copy — mutations won't stick.

Failure modes to watch while coding

- Expecting Python-like always-reference objects — Go structs are values by default
- Mixing value and pointer receivers inconsistently
- Returning pointers to loop variables incorrectly (fixed in modern Go, still understand escapes)

When a test fails, ask which failure mode you hit before changing code at random.

Idioms vs Python pitfalls

- Python objects are references; Go structs are values unless you pointer them.
- Receiver name is convention (`c`), not `self`/`this`.
- No inheritance hierarchy — embed and interface instead.
- Copying a struct copies fields; mutable nested pointers still share.

When you catch yourself reaching for a Python habit, pause and rewrite the Go way once on purpose — even if both would "work". The goal is a Go-shaped intuition.

Going deeper

Large structs passed by value burn CPU copying; profile before micro-optimizing, but prefer *T for mutex-containing structs always.

Practice checklist

Before the exercise, confirm you can:

1. Explain "Structs, Methods, and Pointers" aloud in under two minutes without notes.
2. Re-type the worked example from memory and match the expected output.
3. Name the Go standard library packages touched by this lesson.
4. List one production bug this lesson's rules prevent.
5. Contrast the Go approach with the Python instinct listed above.
6. Write a three-line cheat sheet: one idiom, one pitfall, one command to remember.

If any item fails, re-run the mini-lab once more, then attempt the exercise tests.

Transfer to production

Ask where this shows up outside toys: config loading, API handlers, data pipelines, CI scripts, or on-call debugging. Write one concrete two-sentence scenario tying the worked example to that scenario. Tomorrow, when you rewrite the exercise from memory, check whether the code still fits that framing.

Common mistakes (especially from Python instincts)

- Expecting Python-like always-reference objects — Go structs are values by default
- Mixing value and pointer receivers inconsistently
- Returning pointers to loop variables incorrectly (fixed in modern Go, still understand escapes)

Exercises

Work in `exercises/07-structs-methods-pointers/`. Read that folder's `README.md` before you code.

What you will build

Implement NewCounter, (*Counter).Inc, and Value with pointer receivers.

Study loop

- **A (Drill):** make unit tests pass with the simplest correct code.
- **B (Apply):** extend with an edge case called out in the exercise README.
- **C (Challenge):** rewrite your solution from memory the next day without looking.

Do not open `solutions/07-structs-methods-pointers/` until you have an honest failing attempt.

Self-check

Answer without scrolling up:

1. When should a method use a pointer receiver?
2. What does `&` and `*` mean?
3. What happens if you call a method on a nil pointer with a pointer receiver?

Further reading

- https://go.dev/tour/moretypes
- https://go.dev/doc/effective_go#pointers_vs_values